SIMATS ENGINEERING

ASSESSMENT TOOL 1

Scenario-Based Assessment

Course Name	Internet Programming
Course Code	CSA4307
Course Outcome	CO 3 (L3)
Weightage	40%
Total Marks	30
Faculty	Dr. V. Parthipan

Student Name	K Ganesh
Register Number	192411170

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Answer

a) Architecture of a servlet-based application

A servlet-based application works on the client-server model and normally has three tiers. The first tier is the client, which is the web browser where the student opens the registration form. The second tier is the web server with the servlet container (for example Apache Tomcat), where the servlet runs. The third tier is the database, where the student details are stored.

When the student clicks the submit button, the browser sends an HTTP request to the web server. The request does not go to the servlet directly. The servlet container receives it, finds the servlet that is mapped to that URL, creates the HttpServletRequest and HttpServletResponse objects and calls the servlet. The servlet reads the form data from the request, processes it, and writes the result into the response object. The container then sends this response back to the browser as an HTML page.

Browser (Client)	Servlet Container (Tomcat)	Servlet	Database
Student fills the form and submits it	Receives the request and finds the servlet using the URL mapping	doGet() or doPost() reads the data and processes it	Stores the student details

Flow: Browser → Servlet Container → Servlet → Database, and the response comes back in the reverse order. The URL mapping is given either in the web.xml file or by the @WebServlet annotation. The container also takes care of creating threads, managing the life cycle of the servlet and security, so the programmer only has to write the logic of the servlet.

b) Servlet to handle form data using GET and POST

First the HTML form is created to collect the student details. The method of the form is POST.

register.html

```
<html>
<head><title>Student Registration</title></head>
<body>
  <h2>Student Registration Form</h2>
  <form action="register" method="post">
    Name : <input type="text" name="name"><br><br>
    Register No : <input type="text" name="regno"><br><br>
    Department : <input type="text" name="dept"><br><br>
    Email : <input type="text" name="email"><br><br>
    Password : <input type="password" name="pwd"><br><br>
    <input type="submit" value="Register">
  </form>
</body>
</html>
```

Next the servlet is written. It has the doGet() method for GET requests and the doPost() method for POST requests. Both of them call one common method, so the same output is given for both.

RegisterServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

  public void init() throws ServletException {
    System.out.println("RegisterServlet initialized");
  }

  protected void doGet(HttpServletRequest req, HttpServletResponse res)
      throws ServletException, IOException {
    showDetails(req, res, "GET");
  }
```

```
  protected void doPost(HttpServletRequest req, HttpServletResponse res)
      throws ServletException, IOException {
    showDetails(req, res, "POST");
  }

  private void showDetails(HttpServletRequest req, HttpServletResponse res,
      String method) throws IOException {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();

    String name  = req.getParameter("name");
    String regno = req.getParameter("regno");
    String dept  = req.getParameter("dept");
    String email = req.getParameter("email");
```

```
    out.println("<html><body>");
    out.println("<h3>Registration Successful (" + method + ")</h3>");
    out.println("<p>Name : " + name + "</p>");
    out.println("<p>Register No : " + regno + "</p>");
    out.println("<p>Department : " + dept + "</p>");
    out.println("<p>Email : " + email + "</p>");
    out.println("</body></html>");
  }

  public void destroy() {
    System.out.println("RegisterServlet destroyed");
  }
}
```

With the GET method the details are sent in the URL, for example register?name=Ganesh&regno=192411170, and doGet() is called. With the POST method the details are sent inside the body of the request and doPost() is called. In both cases the servlet reads the values using the getParameter() method.

c) Servlet life cycle

The life cycle of a servlet is controlled by the servlet container. The servlet class is loaded and an object is created, and after that the following three methods are called.

Method	When it is called	Purpose
init()	Only once, when the servlet is created for the first time	Initial work such as reading configuration values or opening a database connection
service()	Every time a request comes from the client, each request runs in a separate thread	Finds the type of the request and calls doGet() or doPost()
destroy()	Only once, when the servlet is removed or the server is stopped	Closes the resources like database connections and files

In the registration servlet, init() is executed when the first student opens the page. After that service() is called for every student who submits the form, and it passes the request to doPost() or doGet(). When Tomcat is shut down, destroy() is called and the servlet is removed from memory.

d) GET or POST for sensitive data

POST is the more suitable method for handling sensitive data. In the registration form the student enters a password and personal details, and these should not be visible to others. With the GET method the data is added to the URL, so it can be seen in the address bar, it is saved in the browser history and it is written in the server log files. With the POST method the data is sent in the body of the request, so it is not shown in the URL.

Another point is that GET has a limit on the length of the URL, but POST can send a large amount of data. Also, a registration inserts a new record, so it is better to use POST because it is meant for operations that change the data on the server. Still, POST alone does not encrypt the data, so HTTPS should be used along with it in a real application. GET is used only for simple requests like searching or viewing a page.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication. Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Answer

a) How session tracking works

HTTP is a stateless protocol, which means the server treats every request as a new one and does not remember the user from the earlier request. In a website the user logs in once and then goes through many pages, so the server must have some way to identify the same user in every request. This is called session tracking.

There are four common ways to do it: cookies, HttpSession, URL rewriting and hidden form fields. In the HttpSession method, the servlet container creates a session object on the server when the user logs in and gives it a unique session ID called JSESSIONID. This ID is sent to the browser, usually as a cookie. In every next request the browser sends the same ID back, the container finds the matching session object, and in this way the server knows that the request is from the same user. If the browser does not accept cookies, the ID can be added to the end of the URL by using URL rewriting.

b) Session management system using HttpSession and Cookies

The system uses three servlets. LoginServlet checks the username and password and creates the session. ProfileServlet is a protected page which opens only if the user has a valid session. LogoutServlet ends the session. A cookie is also used to remember the name of the last user.

LoginServlet.java

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
  protected void doPost(HttpServletRequest req, HttpServletResponse res)
      throws ServletException, IOException {
    String user = req.getParameter("username");
    String pass = req.getParameter("password");

    // demo check is hard coded, normally the database is used
    if (user.equals("ganesh") && pass.equals("1234")) {

      HttpSession session = req.getSession(true);
      session.setAttribute("user", user);
      session.setMaxInactiveInterval(10 * 60);

      Cookie c = new Cookie("lastUser", user);
      c.setMaxAge(60 * 60 * 24);
      res.addCookie(c);

      res.sendRedirect("profile");
    } else {
      res.setContentType("text/html");
      res.getWriter().println("Invalid username or password. "
          + "<a href='login.html'>Try again</a>");
    }
  }
}
```

ProfileServlet.java

```
@WebServlet("/profile")
public class ProfileServlet extends HttpServlet {
  protected void doGet(HttpServletRequest req, HttpServletResponse res)
      throws ServletException, IOException {
    HttpSession session = req.getSession(false);
    if (session == null || session.getAttribute("user") == null) {
      res.sendRedirect("login.html");
      return;
    }
    String user = (String) session.getAttribute("user");
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<h2>Welcome " + user + "</h2>");
    out.println("<p>Session ID : " + session.getId() + "</p>");
    out.println("<a href='logout'>Logout</a>");
  }
}
```

LogoutServlet.java

```
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
  protected void doGet(HttpServletRequest req, HttpServletResponse res)
      throws ServletException, IOException {
    HttpSession session = req.getSession(false);
    if (session != null) {
      session.invalidate();
    }
    res.sendRedirect("login.html");
  }
}
```

In LoginServlet, req.getSession(true) creates a new session and the username is stored in it with setAttribute(). The session is set to expire after 10 minutes of no activity. A cookie named lastUser is also added to the response and it stays in the browser for one day. In ProfileServlet, req.getSession(false) returns the existing session and does not create a new one. If there is no session, the user is sent back to the login page, so nobody can open the profile page just by typing its URL. In LogoutServlet, the invalidate() method removes the session from the server.

c) Comparison of session-based tracking and cookies

Point	HttpSession	Cookies
Place of storage	Stored on the server	Stored in the user's browser
Security	More secure, only the session ID goes to the browser	Less secure, the user can see and change the cookie
Data size	Can store any Java object	Only small text data, about 4 KB
Life time	Ends on logout, time out or when the browser is closed	Can stay for days or months if the max age is set
Server load	Uses server memory for each user	No load on the server
Dependency	Usually depends on a cookie for the session ID	Does not work if the user disables cookies
Used for	Login details, shopping cart	Remember me option, language and theme settings

So, important information like the logged-in user should be kept in HttpSession, and cookies should be used for small and non-sensitive information like the last user name. Both are used together in this system, because the session ID itself is carried by a cookie.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application. Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Answer

a) JDBC architecture

JDBC (Java Database Connectivity) is an API that is used by a Java program to connect to a relational database and execute SQL statements. The architecture has two layers. The upper layer is the JDBC API, which is used by the programmer and contains the classes and interfaces of the java.sql package. The lower layer is the JDBC Driver API, which connects the Java program to the actual database through a database-specific driver.

Component	Description
Java Application	Our program (a servlet or a normal Java class) which has the JDBC code
JDBC API	Interfaces and classes in java.sql such as Connection, Statement, PreparedStatement and ResultSet
DriverManager	Loads the driver and gives the connection to the database
JDBC Driver	Converts the JDBC calls into the protocol of the database (MySQL Connector/J for MySQL)
Database	The MySQL database where the product table is stored

DriverManager manages the drivers and creates the Connection. The Connection object is the link between the program and the database. Statement and PreparedStatement are used to send the SQL queries, and PreparedStatement is better because it is precompiled and protects the program from SQL injection. ResultSet holds the rows that come back from a SELECT query.

b) Steps for database connectivity

1. Import the java.sql package.
2. Load the driver class using Class.forName("com.mysql.cj.jdbc.Driver"). In the newer versions this is done automatically.
3. Create the connection using DriverManager.getConnection(url, username, password).
4. Create a Statement or PreparedStatement object for the SQL query.

5. Execute the query. executeUpdate() is used for INSERT, UPDATE and DELETE, and executeQuery() is used for SELECT.
6. Process the rows returned in the ResultSet.
7. Close the ResultSet, Statement and Connection.

c) JDBC application to insert and retrieve product data

First the database and the product table are created in MySQL.

```
CREATE DATABASE shop;
USE shop;
CREATE TABLE product (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  price DOUBLE,
  quantity INT
);
```

ProductDemo.java

```
import java.sql.*;

public class ProductDemo {
  static final String URL  = "jdbc:mysql://localhost:3306/shop";
  static final String USER = "root";
  static final String PASS = "root";

  public static void main(String[] args) {
    Connection con = null;
    PreparedStatement ps = null;
    ResultSet rs = null;
    try {
      Class.forName("com.mysql.cj.jdbc.Driver");
      con = DriverManager.getConnection(URL, USER, PASS);
      System.out.println("Connected to database");
```

```
      // insert product data
      String ins = "INSERT INTO product VALUES (?, ?, ?, ?)";
      ps = con.prepareStatement(ins);
      ps.setInt(1, 101);
      ps.setString(2, "Wireless Mouse");
      ps.setDouble(3, 599.00);
      ps.setInt(4, 25);
      int rows = ps.executeUpdate();
      System.out.println(rows + " product inserted");

      // retrieve product data
      String sel = "SELECT * FROM product";
      ps = con.prepareStatement(sel);
      rs = ps.executeQuery();
      System.out.println("ID   Name             Price   Qty");
      while (rs.next()) {
        System.out.println(rs.getInt("id") + "   "
            + rs.getString("name") + "   "
            + rs.getDouble("price") + "   "
            + rs.getInt("quantity"));
      }
    } catch (ClassNotFoundException e) {
      System.out.println("Driver not found : " + e.getMessage());
    } catch (SQLIntegrityConstraintViolationException e) {
      System.out.println("Duplicate product id : " + e.getMessage());
    } catch (SQLException e) {
      System.out.println("Database error : " + e.getMessage());
      System.out.println("SQL State : " + e.getSQLState());
    } finally {
      try {
        if (rs != null)  rs.close();
        if (ps != null)  ps.close();
        if (con != null) con.close();
      } catch (SQLException e) {
        System.out.println("Error while closing : " + e.getMessage());
      }
    }
  }
}
```

Output

```
Connected to database
1 product inserted
ID   Name             Price   Qty
101   Wireless Mouse   599.0   25
```

d) Exception handling

A database program can fail for many reasons, like a wrong password, the database server not running, a wrong SQL statement or a duplicate primary key. So the code is written inside a try block and the errors are caught by the catch blocks. ClassNotFoundException is caught when the driver jar file is not added to the project. SQLIntegrityConstraintViolationException is caught when a duplicate id is inserted. SQLException is the main exception for all other database errors, and its methods getMessage() and getSQLState() give the details of the problem.

The finally block is always executed, whether there is an error or not. It is used to close the ResultSet, PreparedStatement and Connection, so there is no connection leak. Using PreparedStatement with ? placeholders also stops SQL injection. In this way the e-commerce application can store the product details in the database and read them back safely.

Code of Conduct

I K Ganesh / 192411170 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Ganesh

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand